Espresso AI 3-Class (*EAI-3Class*) Multiclass logistic regression classifier

Espresso AI Contest

Submitter Information

Name : Corrado Vaccaro, Maria Di Benedetto

Email : corrado.vaccaro@nokia.com, maria.di_benedetto@nokia.com

Organization : NI S&D EUR East Med, TCA

Espresso AI Contest

Use Case Information

Use Case Name*: EAI-3Class

Abstract: *This work presents a supervised multiclass predictive model that replaces rigid deterministic rules with a probabilistic, data-driven methodology capable of handling heterogeneous and incomplete real-world datasets. By learning from diverse key performance indicators provided in the data set, the model automatically categorizes mobile-network sites and can assist in several use cases such as the early detection of potential faults.*

(*) as created in DataQuestor tool

Espresso AI Contest

Use Case Scope

Objective

- Develop a supervised multiclass predictive classification model capable of estimating the class membership of each observation, overcoming the limitations of approaches based solely on deterministic formulas.

Rationale

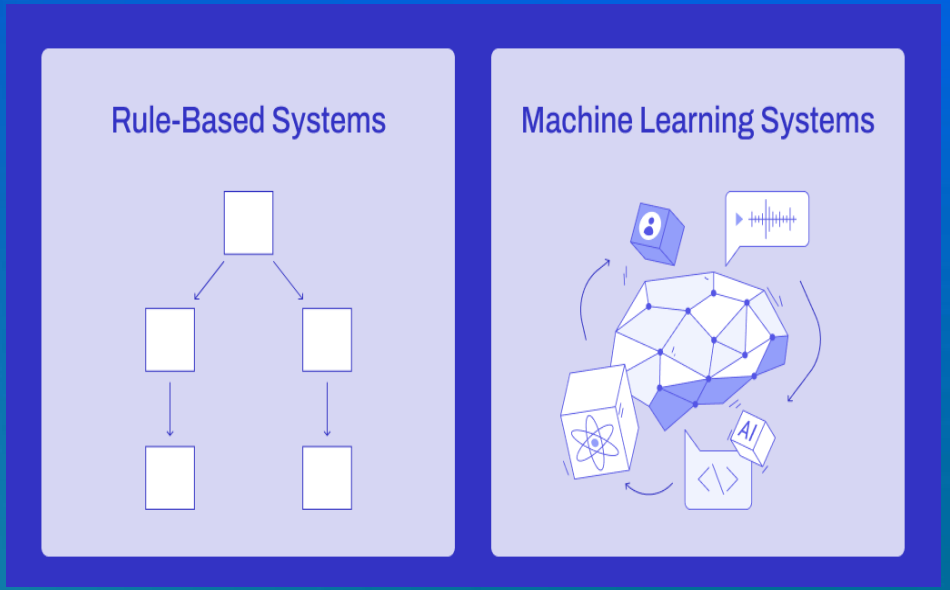
- Deterministic formulas are effective only when all required variables are available, complete, and reliable. In real-world contexts, data are often heterogeneous and noisy and include both numerical and categorical variables that cannot be exhaustively captured by a single analytical rule.

Implemented Predictive System

- Combines numerical and categorical variables through a structured preprocessing pipeline;
- Handles incomplete data via dedicated data-quality checks and preprocessing steps;
- Learns latent relationships and patterns from historical data that cannot be explicitly modeled by deterministic rules;
- Produces probabilistic class estimates, which are used to derive the final predicted class.

Outcome

- A rigid decision rule is replaced by a generalizable predictive model, validated on training and validation datasets and designed for use in real, dynamic operational scenarios, ensuring methodological consistency and operational robustness.



Espresso AI Contest

Use Case Detailed Description

Objective

- *EAI-3Classis* a single-layer neural classifier that combines multiple KPI and contextual features to estimate the probability of each performance class (Good, Medium, Poor).

Dataset

- Source: “KPI_set2”: 6 KPIs from “CIB_Network@20250930” (from MUSA Project “CIB_for_Espresso”)
- Period of time: 2025-09-22 to 2025-10-06 (15 days)
- Size: 5.933.520 records
- Features: 10 variables
- Useful for training
  - 6 KPI
  - 4 contextual features
- Target class
  - performance class (Good, Medium, Poor)

Model Goals

- Multiclass logistic regression computes one linear score per class, transforms them into probabilities using a *softmax* function, and assigns the class with the highest probability via *argmax*
- Visualize training/validation metrics and *confusion* matrix

Development environment

- Machine: Apple MacBook Pro M5 24Gbyte
- Framework: JupyterLab 4.5.0, Msty Studio 2.2.1
- Language: Python 3.11
- Libraries: NumPy, Pandas, Sklearn, Matplotlib, Joblib
- Collaborator Model: *gpt-oss-20b-MXFP4-Q8* (MLX), *Qwen2.5 7B* (Ollama) via Msty Studio (**LOCAL** implementation), Nokia_GPT

Bibliography and Inspiration:

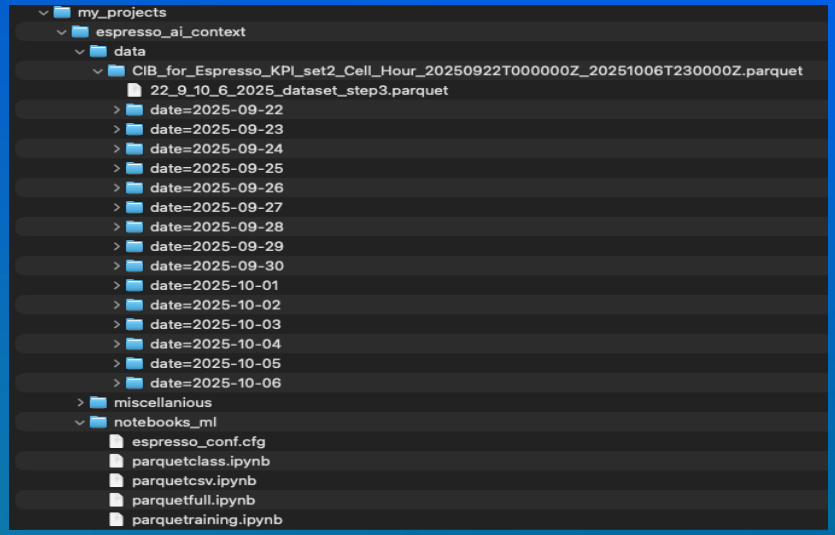
- *Python and Machine Learning* A. Bellini, A. Guidi, McGraw-Hill.
- *The Machine Learning Gym* A. Metelli, F. Trovò , McGraw-Hill.
- Deep Learning, S. Weidman, Tecniche Nuove
- *Diabetes Health 3-Classificator (3-DHC) a Neural Network for Diabetes Risk Prediction*, Corrado Vaccaro, Proof of Concept for early diabetes detection

Espresso AI Contest

Application and detailed Run Instruction

Python Application* Name: parquetcsv, parquetclass, parquetfull, parquetraining
Run instruction:

- Create a project structure as indicated in the picture
- Under *data* directory there is *CIB_for_Espresso_KPI_set2_Cell_Hour_20250922T000000Z_20251006T230000Z.parquet* directory containing all dataset (single day into dedicated directory)
- Into *notebooks_ml* directory create configuration file named *espresso_conf.cfg* containing the section PATH where BASE_PATH is defined and contains the path indicating *CIB_for_Espresso_KPI_set2_Cell_Hour_20250922T000000Z_20251006T230000Z.parquet* directory
- All notebooks .ipynb are located into *notebooks_ml* directory
- Create a virtual environment for Python 3.11 where all libraries and Jupyter are installed
- Execute Jupyter
- In the following sequence execute:
  - parquetcsv – Preprocessing Step 1
  - Parquetclass – Preprocessing Step 2
  - Parquetfull – Preprocessing Step 3
  - Parquetraining – Training model implementation



Attach as object Source Code package here:

- parquetcsv.ipynb, parquetclass.ipynb, parquetfull.ipynb, parquetraining.ipynb
- espresso_conf.cfg

Espresso AI Contest

Input data and pre-processing – Step 1 (parquetcsv)

Dataset

- Source: “KPI_set2”: 6 KPIs from “CIB_Network@20250930” (from MUSA Project “CIB_for_Espresso”)
- Period of time: 2025-09-22 to 2025-10-06 (15 days)
- Size: 5.933.520 records

Features: 10 variables

- Useful for training
  - Numerical KPI Inputs : volte_cssr, 98_RRC_setup_Success_ratio, 113_E_RAB_Setup_Success_Rate_All, 110_E_RAB_Blocking_Rate, 141_DL_Packet_Loss_Rate_PDPC, 142_UL_Packet_Loss_Rate_PDPC
  - Static Categorical Inputs : OSS Site Name, OSS Class, Vendor, Technology

Purpose of Step 1 – Data cleaning and temporal normalization

- reduce the dataset to only the required columns, including selected KPI metrics, static metadata (vendor, technology, cell class, site), the unique cell identifier (name)
- reconstruct a synthetic but coherent timestamp (*datetime_fixed*) by assigning exactly 24-hourly instances (0–23) to each (cell, day) pair
- produces, for each day, a <date>_step1.csv file containing raw KPI values, static metadata, a reconstructed and consistent timestamp

NOKIA

Espresso AI Contest

Input data and pre-processing – Step 2 (parquetclass)

Purpose of Step 2 – Definition of the classification function (labeling)

- Its purpose is to formally define and implement the deterministic function that maps the observed KPIs into a performance class. The performance class is obtained by classifying each KPI using thresholds, combining the classes with a weighted score, and mapping the aggregate score into a final Good/Medium/Poor class.
- Step 2 explicitly establishes:

$$f: (KPI_1, KPI_2, \dots, KPI_n) \rightarrow \text{Good, Medium, Poor}$$

- This function constitutes the supervised ground truth on which the ML model will be trained.
- The function is composed of three deterministic transformations:
  - Classification of individual KPIs,
  - KPI class → numerical score (weighted score)
  - Aggregate score → final target class

Espresso AI Contest

Input data and pre-processing – Step 2 (parquetclass)

- Classification of individual KPIs
  - Each KPI is individually classified into: $KPI\_i \rightarrow C\_i \in \{\text{"Good"}, \text{"Medium"}, \text{"Poor"}\}$ using fixed thresholds (NaN, out-of-range, or invalid values → Poor)*:

Classe	110_E_RAB_Blocking_Rate
Good	< 0.1
Medium	>= 0.1 e <= 0.5
Poor	> 0.5

Classe	141_DL_Packet_Loss_Rate_PDPC
Good	< 0.05
Medium	>= 0.05 e <= 0.1
Poor	> 0.1

Classe	142_UL_Packet_Loss_Rate_PDPC
Good	< 0.05
Medium	>= 0.05 e <= 0.1
Poor	> 0.1

Class	volte_cssr
Good	> 99.0
Medium	>= 98.5 e <= 99.0
Poor	< 98.5

Class	98_RRC_setup_Success_ratio
Good	> 99.0
Medium	>= 98.0 e <= 99.0
Poor	< 98.0

Classe	113_E_RAB_Setup_Success_Rate_All
Good	> 99.9
Medium	>= 99.5 e <= 99.9
Poor	< 99.5

(*) NokiaGPT queried to provide KPI description, Typical Target Value, Performance Interpretation & Troubleshooting

Espresso AI Contest

Input data and pre-processing – Step 2 (parquetclass)

- KPI class → numerical score (weighted score)
  - Each class is mapped to a score:

Classe	KPI Score
Good	2
Medium	1
Poor	0

  - Each KPI has a weight that reflects its relevance:

KPI	Weight
volte_cssr	2.0
98_RRC_setup_Success_ratio	1.5
113_E_RAB_Setup_Success_Rate_All	1.5
110_E_RAB_Blocking_Rate	1.0
141_DL_Packet_Loss_Rate_PDPC	1.0
142_UL_Packet_Loss_Rate_PDPC	1.0

Espresso AI Contest

Input data and pre-processing – Step 2 (parquetclass)

- KPI class → numerical score (weighted score)
  - The aggregate score is:

$$\text{score} = \sum_i \left(\text{weight}_i \times \text{score}(C_i)\right)$$

  - The maximum possible score is:

$$\text{score}_{max} = 2 \times \sum_i \text{weight}_i$$

- Aggregate score → final target class
  - The score is normalized:

$$\text{score}_{norm} = \frac{\text{score}}{\text{score}_{max}}$$

  - and mapped into the final class:

Final class	Condition
Good	score_norm ≥ 0.80
Medium	0.50 ≤ score_norm < 0.80
Poor	score_norm < 0.50

NOKIA

Espresso AI Contest

Input data and pre-processing – Step 2 (parquetclass)

- Aggregate score → final target class
  - The function produces target_class ∈ {Good, Medium, Poor}for each hourly instance of each cell.
  - The function is: deterministic, reproducible, handles conflicting KPIs, robust to noise near thresholds, and suitable for approximation by multiclass logistic regression.
  - The final dataset - for each day, a <date>_step2.csv contains a single new column, target_class, which represents the performance class calculated deterministically from the KPIs.

Espresso AI Contest

Input data and pre-processing – Step 3 (parquetfinal)

Purpose of Step 3 – Final dataset

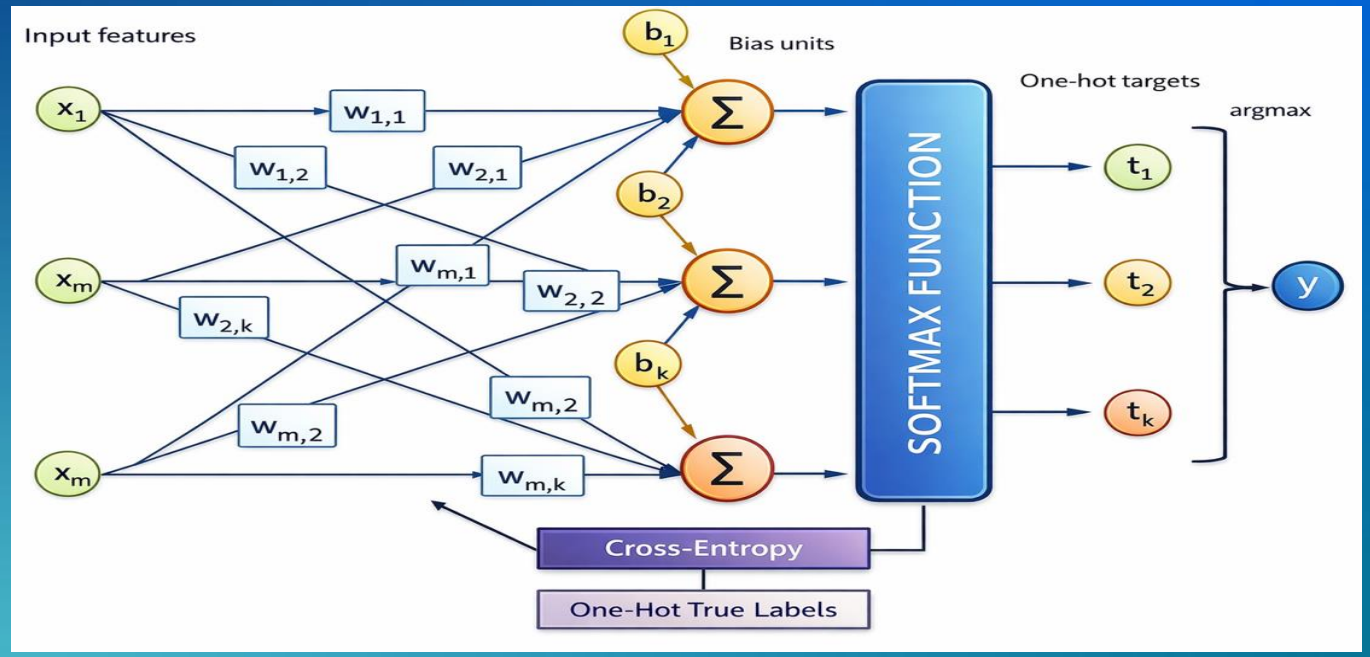
- The purpose of this step is simply to create the final dataset, called 22_9_10_6_2025_dataset_step3.parquet, by aggregating all the <data>_step2.csv files

Espresso AI Contest

Multiclass Logistic Regression Architecture

Multiclass Logistic Regression (Softmax Classifier)

- The model is a single-layer neural classifier that combines multiple KPI and contextual features to estimate the probability of each performance class (Good, Medium, Poor). Multiclass logistic regression computes one linear score per class, transforms them into probabilities using a *softmax* function, and assigns the class with the highest probability via *argmax*.



Espresso AI Contest

Multiclass Logistic Regression Architecture

- Input Layer
  - Numerical KPI Inputs: RRC Setup Success Ratio, E-RAB Blocking Rate, E-RAB Setup Success Rate (All), DL Packet Loss Rate (PDCP), UL Packet Loss Rate (PDCP), VoLTE CSSR
  - Static Categorical Inputs: OSS Site Name, OSS Class, Vendor, Technology (4G)
- Preprocessing Layer
  - Numerical branch: Median Imputation, Standard Scaling
  - Categorical branch: Most Frequent Imputation, One-Hot Encoding
- Linear Combination Layer
  - For each class k:

$$z_k = w_k^T x + b_k$$

where: x $= preprocessed\ feature\ vector$, $w_k = weight\ vector\ for\ class\ k$, $b_k = bias\ term$

This layer is fully connected, exactly like a neural network layer without hidden layers.

- Softmax Output Layer

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- Output neurons
  - Good Performance, Medium Performance, Poor Performance
  - Each neuron outputs a probability.
- Final prediction
  - Predicted class = *argmax* probability.
- Loss function (Cross-Entropy)
  - $\mathcal{L}(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log(\widehat{y_k})$
  - $y_k \in \{0,1\}$ is the target one-hot
  - $\widehat{y_k} = P(y = k \mid x)$ is the probability predicted by the softmax
  - $\mathcal{L} = -\log(\widehat{y_c})$ (according to the model)

Espresso AI Contest

Multiclass Logistic Regression Architecture

This section presents a simple but complete numerical example illustrating the full computation flow of a Multiclass Logistic Regression (Softmax Classifier), consistent with the model used in the project, but applied to a reduced dataset

- We consider: 5 instances, 5 numerical features, 3 classes: Good, Medium, Poor and *Softmax* + *argmax* decision rule
- Input dataset, 10 instances x 5 features (Each row represents a single observation $x^{\{(i)\}}$
- Each class k is associated with a weight w_k and a bias b_k learned during the training
  - Good w_G =[1.2, 0.8, −0.6, −0.5, 1.0],b_G=0.3
  - Medium w_M =[0.3, 0.5, 0.4, 0.6, 0.2],b_M=0.1
  - Poor w_P =[−0.8, −0.6, 1.2, 1.1, −0.5],b_P=−0.2
- For each instance $x = [0.8,\ 1.2,\ 0.1,\ 0.4,\ 1.0]$ the model computes one linear score per class
  - Good $z_G = 2{,}92$, Medium $z_M = 1.47$, Poor $z_P = -1.31$

ID	f_1	f_2	f_3	f_4	f_5
1	0.8	1.2	0.1	0.4	1.0
2	1.1	0.9	0.2	0.3	0.8
3	0.2	0.4	1.5	1.2	0.3
4	0.3	0.6	1.3	1.1	0.5
5	1.5	1.1	0.2	0.2	1.3

Espresso AI Contest

Multiclass Logistic Regression Architecture

- The logits are converted into probabilities using the softmax function:

$$softmax(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- For instance 1

Class	z	exp(z)	Probability
Good	2.92	18.54	0.80
Medium	1.47	4.35	0.19
Poor	-1.31	0.27	0.01

- The final prediction is obtained by selecting the class with the highest probability:

$$\hat{y} = \arg\max_k P(y = k)$$

- max(0.80, 0.19, 0.01) → Good

Espresso AI Contest

Training Process

- The training of the *EAI-3Class* neural network occurs through two main phases: pipeline preparation and pipeline training cycle
  - Pipeline preparation.
    - Scikit-learn pipeline allows you to chain multiple transformations (and/or a final model) in an ordered sequence
    - For numeric features, all NaNs are replaced with the column median calculated on the training data
    - For categorical features, it replaces NaNs with the most frequent value in the column (the mode)
    - Apply different transformations to different columns and concatenate the results into a single feature matrix
  - Pipeline training *LogisticRegression* cycle with following parameters:
    - *LogisticRegression* : it is a linear model for binary or multiclass classification, which estimates the probability of class membership using the logistic function
    - *class_weight* = 'balanced' : it is used to manage unbalanced classes
    - *solver* = 'saga' : is the numerical algorithm used to optimize the cost function. It is suitable for large and sparse datasets
    - *max_iter* = 300 : maximum number of solver iterations. If the model doesn't converge first, it stops at 300 iterations
    - *tol* = 1e-3 : tolerance for the stopping criterion. The solver stops when the improvement of the loss function, ΔL , is less than *tol,* where $\Delta L = \left|L^{(k-1)} - L^{(k)}\right|$ and $L^{(k)} = \text{value of the loss at k iteraction}$.

NOKIA

Espresso AI Contest

Achievements and Results

- The training was performed on the first thirteen days of the dataset (5142384 instances) while the validation was performed on the remaining two days (791136 instances)
- Core Metrics (Training vs Validation)

Metric	Training	Validation	Interpretation
F1 – Good	0.85	0.84	Strong and stable performance on the main class
F1 – Medium	0.75	0.74	Consistent behavior on the intermediate class
F1 – Poor	0.66	0.62	Most challenging and least represented class
Macro F1	0.76	0.74	Class balance and fairness
Weighted F1	0.79	0.78	Overall real-world performance
Accuracy (secondary)	0.79	0.78	Consistent with weighted F1

- F1-Score
  - *How well does the model handle this class, accounting for both false positives and false negatives?*
  - The model demonstrates strong and stable performance on the “Good” class, maintains solid and consistent results on the “Medium” class, and exhibits expected limitations on the “Poor” class due to data imbalance. The close alignment between training and validation F1-scores indicates good generalization, with no evidence of overfitting and reliable behavior within the constraints of the available data.
- Macro F1-Score
  - *Does the model treat all classes fairly, or does it favors some classes over others?*
  - Macro F1-scores of 0.76 on the training set and 0.74 on the validation set indicate that all classes contribute equally to the evaluation of the model, regardless of their frequency, with no class being systematically ignored, thereby confirming overall balanced learning.
- Weighted F1-Score
  - *How well does the model perform overall on the real-world data distribution?*
  - The Weighted F1-score, which measures overall model performance while accounting for the true class distribution, reaches 0.79 on the training set and 0.78 on the validation set, indicating strong and stable performance with good generalization and no evidence of overfitting.

Espresso AI Contest

Achievements and Results

- Evaluation Metrics – Operational Definitions

Metric	What it measures	How it is computed
F1-score	The overall quality of the model on a class, considering all relevant errors	$\frac{2 \cdot TP}{2 \cdot TP + FP + FN}$
Macro F1-score	The average model performance treating all classes equally	$\frac{1}{K}\sum_{k=1}^{K} F1_k$
Weighted F1-score	The overall model performance weighted by the true class distribution	$\sum_{k=1}^{K} \frac{N_k}{N} \cdot F1_k$
Accuracy (secondary metric)	The overall proportion of correct predictions	$\frac{TP + TN}{TP + TN + FP + FN}$

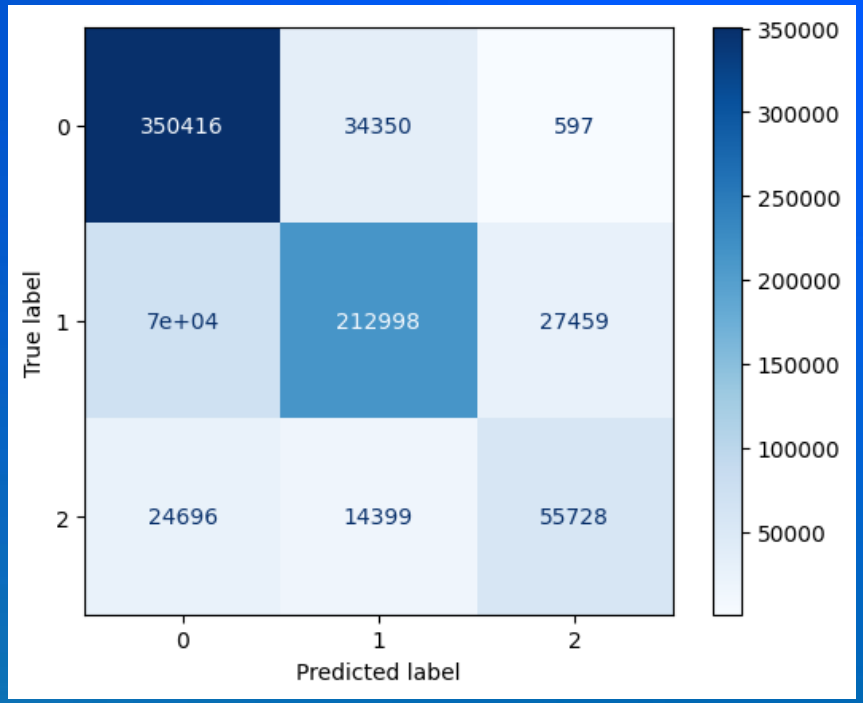
- **TP** = True Positive, **FP** = False Positive, **FN** = False Negative, **TN** = True Negative

NOKIA

Espresso AI Contest

Achievements and Results

- Confusion Matrix – A confusion matrix is a table that compares:
  - actual class (ground truth) → what is correct according to the labeled dataset
  - model-predicted class → what the model estimated
- Structure
  - The main diagonal represents correct predictions
    - Diagonal dominant
  - Off-diagonal values represent classification errors
    - Adjacent errors (Good↔Medium, Medium↔Poor)



True \ Pred	Good (0)	Medium (1)	Poor (2)
Good (0)	350 416	34 350	597
Medium (1)	70 000	212 998	27 459
Poor (2)	24 696	14 399	55 728

Espresso AI Contest

Conclusion – Overall Assessment

- The supervised multiclass predictive model demonstrates that a probabilistic, data-driven approach can effectively replace rigid deterministic rules, even when faced with heterogeneous and incomplete real-world datasets.
- The model shows good overall performance, as evidenced by the clear dominance of the main diagonal in the confusion matrix. Errors are mostly concentrated between adjacent classes (Good ↔ Medium, Medium ↔ Poor), indicating a structural difficulty in separating borderline cases rather than gross classification errors

Espresso AI Contest

Conclusion - Future improvements and developments

- Improve the model on the adjacent classes in order to reduce these cases and increase the values of the main diagonal
- Implement a dedicated application to make new predictions, using the saved model and giving in input list of cells
- Include the predictive model in Assurance Systems.

- Possible use cases of the model include for example:

    - SITE ANALYSIS FAULT PREDICTION - by categorizing mobile sites, the model not only streamlines fault detection but also enables proactive maintenance strategies, ultimately enhancing network reliability and reducing operational costs.
    - CAPACITY FORECASTING - The model can learn traffic-pattern trends from historical KPI data for each site category, automatically accounting for daily, weekly and event-driven seasonality. This way it is possible to predict future resource utilization, and set alerts that trigger proactive actions before congestion impacts users
    - LOAD BALANCING - Using the same probabilistic outputs, the model can estimate the likelihood of overload for each site in the upcoming interval.


NOKIA